

Project Report: Modular Stateful Todo Application Screen

GitHub Link: <https://github.com/SudarshanDevadiga/Todo-List-App-with-State>

Executive Summary

This project demonstrates state management, interactive UI workflows, and modular architectural design within the Flutter framework. The application implements a fully functional Todo list supporting stateful task creation, completion toggling, and item deletion. The codebase is partitioned across two distinct modular files (`main.dart` and `todo_screen.dart`), showcasing runtime state manipulation via `StatefulWidget` and `setState`, modal UI overlays, and list rendering using standard Material layout widgets.

Project Architecture & File Organization

The application adheres to clean separation of concerns by isolating application configuration and entry-point setup from task management presentation and state logic:

Plaintext

lib/
├── `main.dart` # Program Entry Point, MaterialApp Root & Global Theme Configuration
└── `todo_screen.dart` # Task Data Model, Stateful UI Logic & Dynamic List Rendering

Module-by-Module Technical Breakdown

Module 1: Entry Point & Application Configuration (`main.dart`)

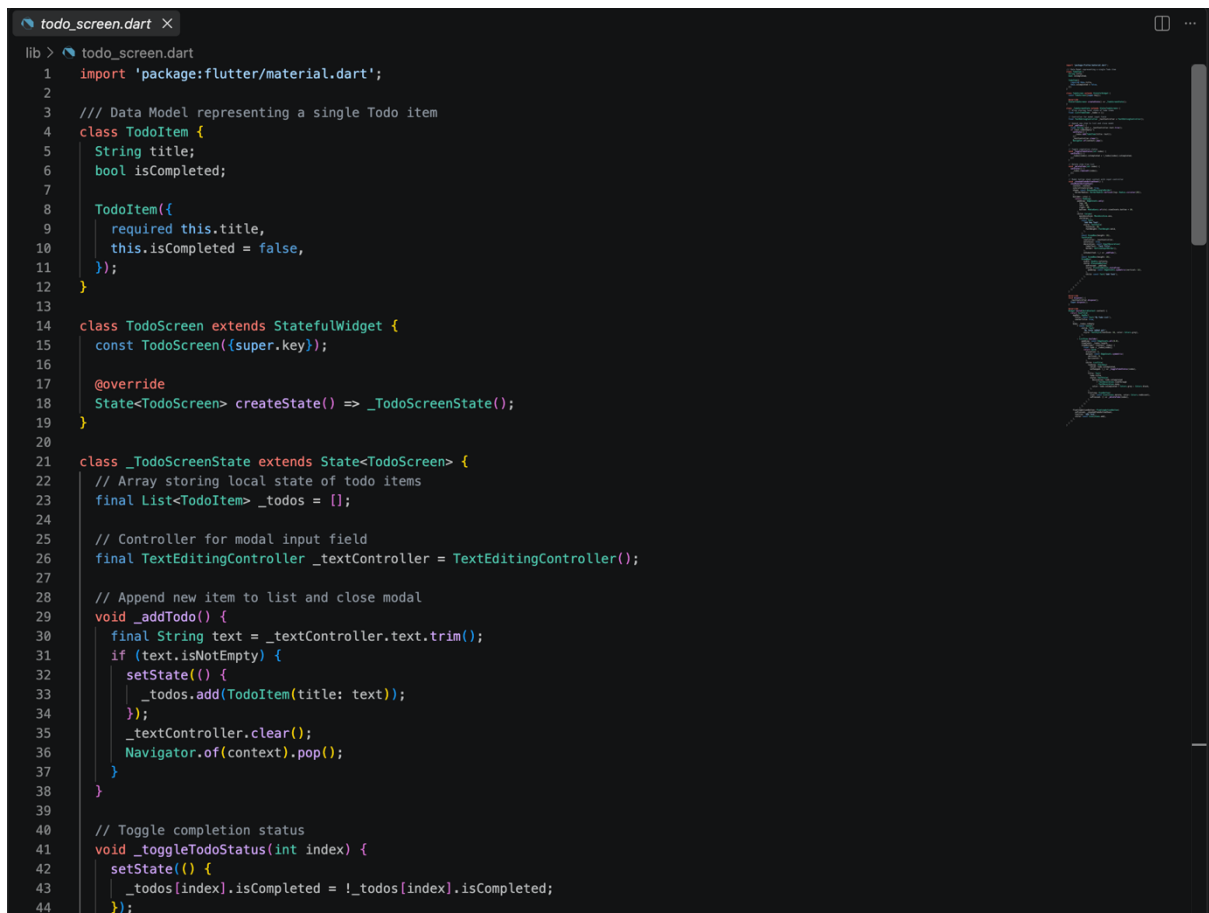
- **Entity:** `MyApp` (`StatelessWidget`)
- **Key Widgets:** `MaterialApp`
- **Mechanics:**
 - Establishes the application entry point using `void main() => runApp(const MyApp());`.
 - Configures global styling via `ThemeData` utilizing `useMaterial3: true` and an indigo primary swatch.
 - Hides the debug banner (`debugShowCheckedModeBanner: false`) and sets `TodoScreen` as the root screen destination.



```
main.dart
lib > main.dart
1 import 'package:flutter/material.dart';
2 import 'todo_screen.dart';
3
4 void main() {
5   runApp(const MyApp());
6 }
7
8 class MyApp extends StatelessWidget {
9   const MyApp({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return MaterialApp(
14       title: 'Todo List App',
15       debugShowCheckedModeBanner: false,
16       theme: ThemeData(
17         primarySwatch: Colors.indigo,
18         useMaterial3: true,
19       ),
20       home: const TodoScreen(),
21     );
22   }
23 }
```

Module 2: Task Model & Stateful UI Presentation (todo_screen.dart)

- **Entities:** TodoItem (Data Class), TodoScreen (StatefulWidget), _TodoScreenState (State)
- **Key Widgets:** Scaffold, AppBar, ListView.builder, Card, ListTile, Checkbox, IconButton, FloatingActionButton, TextField, Column
- **Mechanics:**
 - **Data Modeling:** Defines TodoItem to encapsulate task attributes (title, isCompleted).
 - **State Management:** Maintains an internal List<TodoItem> array updated through setState() triggers on task addition, deletion, and toggle operations.
 - **Dynamic ListView Rendering:** Renders task items using ListView.builder for efficient memory management, displaying an empty state indicator when no tasks exist.
 - **Interactive Overlay:** Employs showModalBottomSheet with a TextEditingController to accept user input for task additions.



```
lib > todo_screen.dart
1 import 'package:flutter/material.dart';
2
3 /// Data Model representing a single Todo item
4 class TodoItem {
5   String title;
6   bool isCompleted;
7
8   TodoItem({
9     required this.title,
10    this.isCompleted = false,
11  });
12 }
13
14 class TodoScreen extends StatefulWidget {
15   const TodoScreen({super.key});
16
17   @override
18   State<TodoScreen> createState() => _TodoScreenState();
19 }
20
21 class _TodoScreenState extends State<TodoScreen> {
22   // Array storing local state of todo items
23   final List<TodoItem> _todos = [];
24
25   // Controller for modal input field
26   final TextEditingController _textController = TextEditingController();
27
28   // Append new item to list and close modal
29   void _addTodo() {
30     final String text = _textController.text.trim();
31     if (text.isNotEmpty) {
32       setState(() {
33         _todos.add(TodoItem(title: text));
34       });
35       _textController.clear();
36       Navigator.of(context).pop();
37     }
38   }
39
40   // Toggle completion status
41   void _toggleTodoStatus(int index) {
42     setState(() {
43       _todos[index].isCompleted = !_todos[index].isCompleted;
44     });
45   }
46 }
```

```

45     }
46
47     // Delete item from list
48     void _deleteTodo(int index) {
49       setState(() {
50         _todos.removeAt(index);
51       });
52     }
53
54     // Modal bottom sheet context with input controller
55     void _showAddTodoBottomSheet() {
56       showModalBottomSheet(
57         context: context,
58         isScrollControlled: true,
59         shape: const RoundedRectangleBorder(
60           borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
61         ),
62         builder: (ctx) {
63           return Padding(
64             padding: EdgeInsets.only(
65               top: 20,
66               left: 20,
67               right: 20,
68               bottom: MediaQuery.of(ctx).viewInsets.bottom + 20,
69             ),
70             child: Column(
71               mainAxisAlignment: MainAxisAlignment.min,
72               children: [
73                 const Text(
74                   'Add New Task',
75                   style: TextStyle(
76                     fontSize: 18,
77                     fontWeight: FontWeight.bold,
78                   ),
79                 ),
80                 const SizedBox(height: 15),
81                 TextField(
82                   controller: _textController,
83                   autofocus: true,
84                   decoration: const InputDecoration(
85                     labelText: 'Task Title',
86                     border: OutlineInputBorder(),
87

```

```

88                   onSubmit: (_) => _addTodo(),
89                 ),
90                 const SizedBox(height: 15),
91                 SizedBox(
92                   width: double.infinity,
93                   child: ElevatedButton(
94                     onPressed: _addTodo,
95                     style: ElevatedButton.styleFrom(
96                       padding: const EdgeInsets.symmetric(vertical: 12),
97                     ),
98                     child: const Text('Add Task'),
99                   ),
100                 ),
101               ],
102             ),
103           );
104         },
105       );
106     }
107
108     @override
109     void dispose() {
110       _textController.dispose();
111       super.dispose();
112     }
113
114     @override
115     Widget build(BuildContext context) {
116       return Scaffold(
117         appBar: AppBar(
118           title: const Text('My Todo List'),
119           centerTitle: true,
120         ),
121         body: _todos.isEmpty
122           ? const Center(
123             child: Text(
124               'No tasks added yet!',
125               style: TextStyle(fontSize: 16, color: Colors.grey),
126

```

```
127     )
128     : ListView.builder(
129       padding: const EdgeInsets.all(8.0),
130       itemCount: _todos.length,
131       itemBuilder: (context, index) {
132         final todo = _todos[index];
133         return Card(
134           elevation: 2,
135           margin: const EdgeInsets.symmetric(
136             vertical: 6,
137             horizontal: 4,
138           ),
139           child: ListTile(
140             leading: Checkbox(
141               value: todo.isCompleted,
142               onChanged: (_) => _toggleTodoStatus(index),
143             ),
144             title: Text(
145               todo.title,
146               style: TextStyle(
147                 decoration: todo.isCompleted
148                   ? TextDecoration.lineThrough
149                   : TextDecoration.none,
150                 color: todo.isCompleted ? Colors.grey : Colors.black,
151               ),
152             ),
```

```
153             trailing: IconButton(
154               icon: const Icon(Icons.delete, color: Colors.redAccent),
155               onPressed: () => _deleteTodo(index),
156             ),
157           ),
158         );
159       },
160     ),
161     floatingActionButton: FloatingActionButton(
162       onPressed: _showAddTodoBottomSheet,
163       tooltip: 'Add Task',
164       child: const Icon(Icons.add),
165     ),
166   );
167 }
168 }
```

Widget Role Breakdown

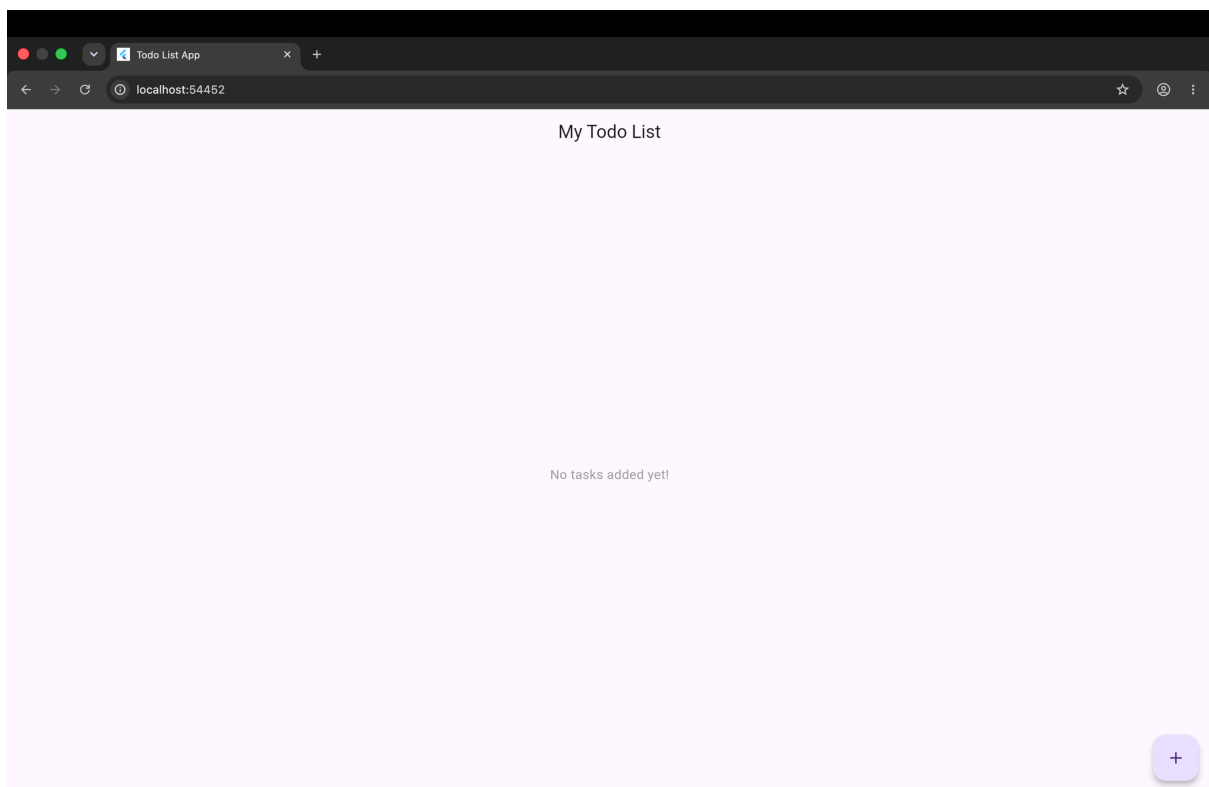
Widget	Role & Implementation in Project
StatefulWidget / State	Manages ephemeral state lifecycle and triggers UI rebuilds via setState() calls.
ListView.builder	Memory-efficient scrollable collection renderer that constructs visible task cards dynamically.
Card & ListTile	Wraps individual items with elevated surface container styling, leading actions, and structural titles.
Checkbox	Toggles individual task completion state (isCompleted), updating visual line-through attributes.
IconButton	Provides tap interaction handlers bound to _deleteTodo() for task deletion.

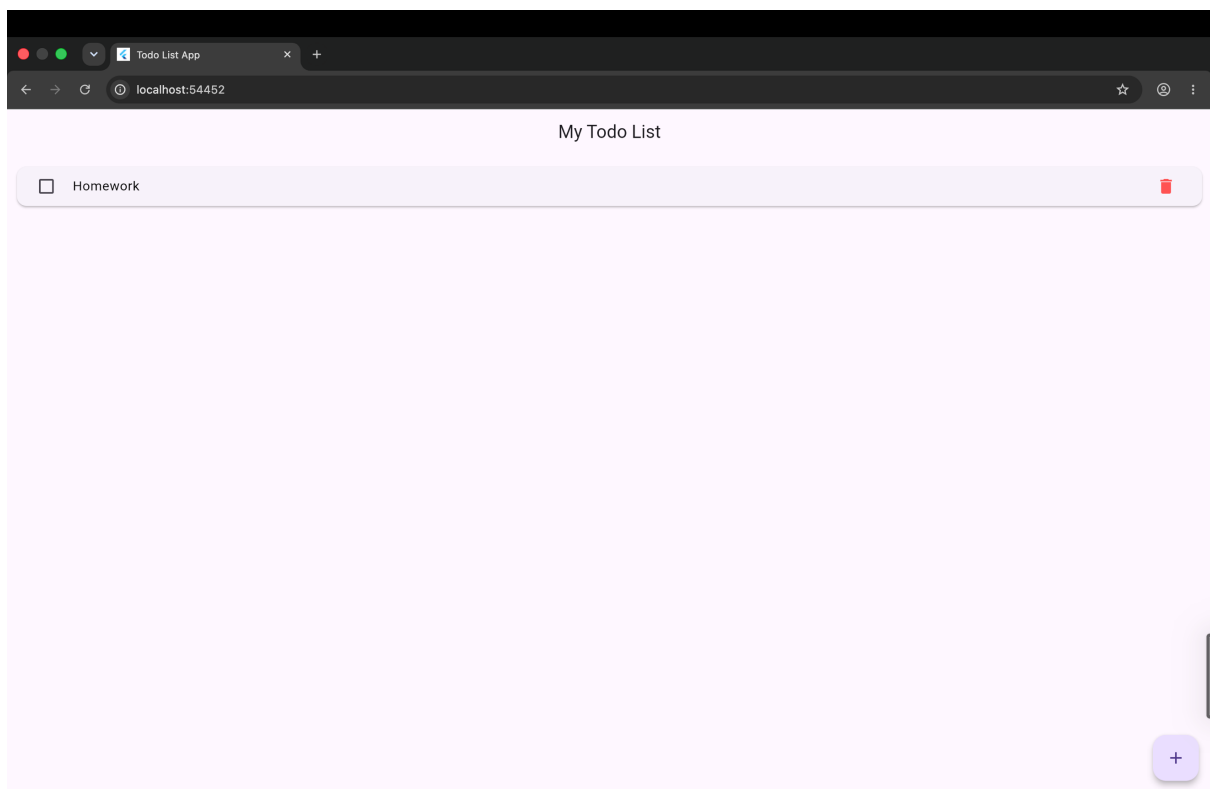
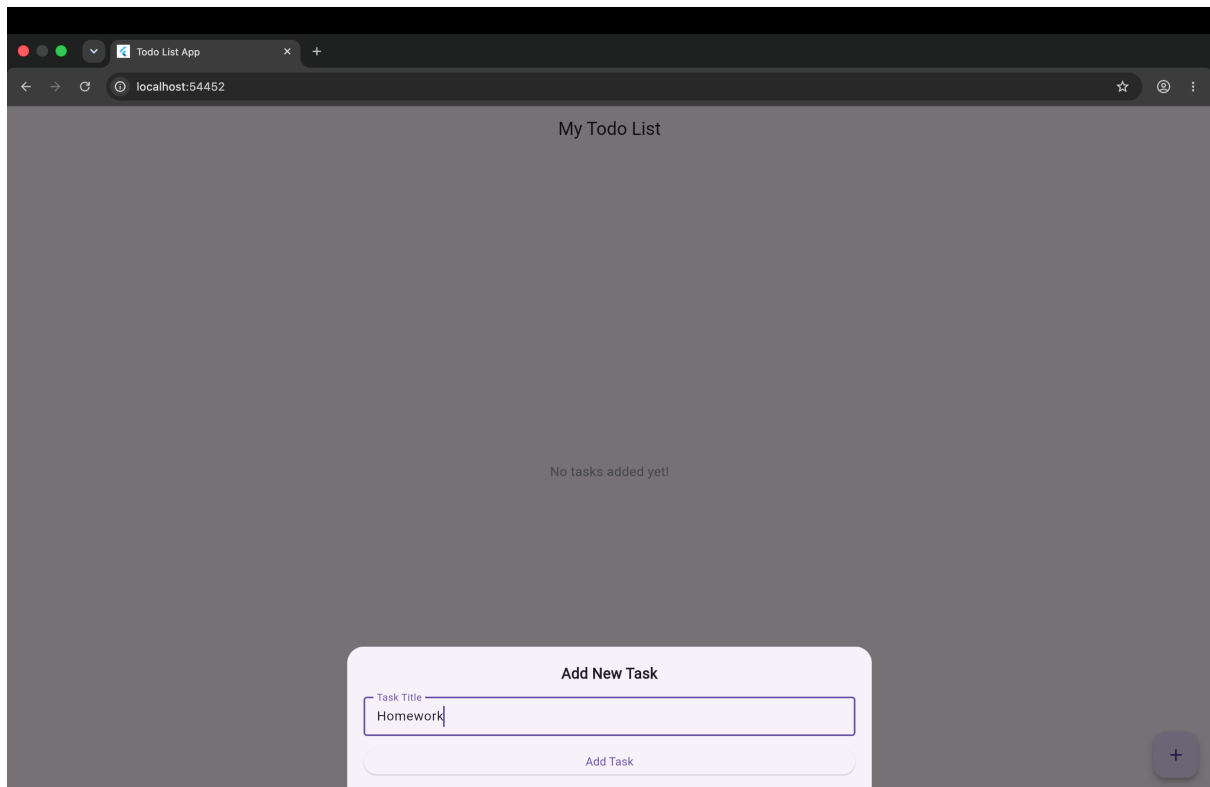
Widget	Role & Implementation in Project
TextField	Captures string inputs within modal dialogs using an explicit TextEditingController.
FloatingActionButton	Main call-to-action trigger anchored at bottom-right to open the task creation sheet.

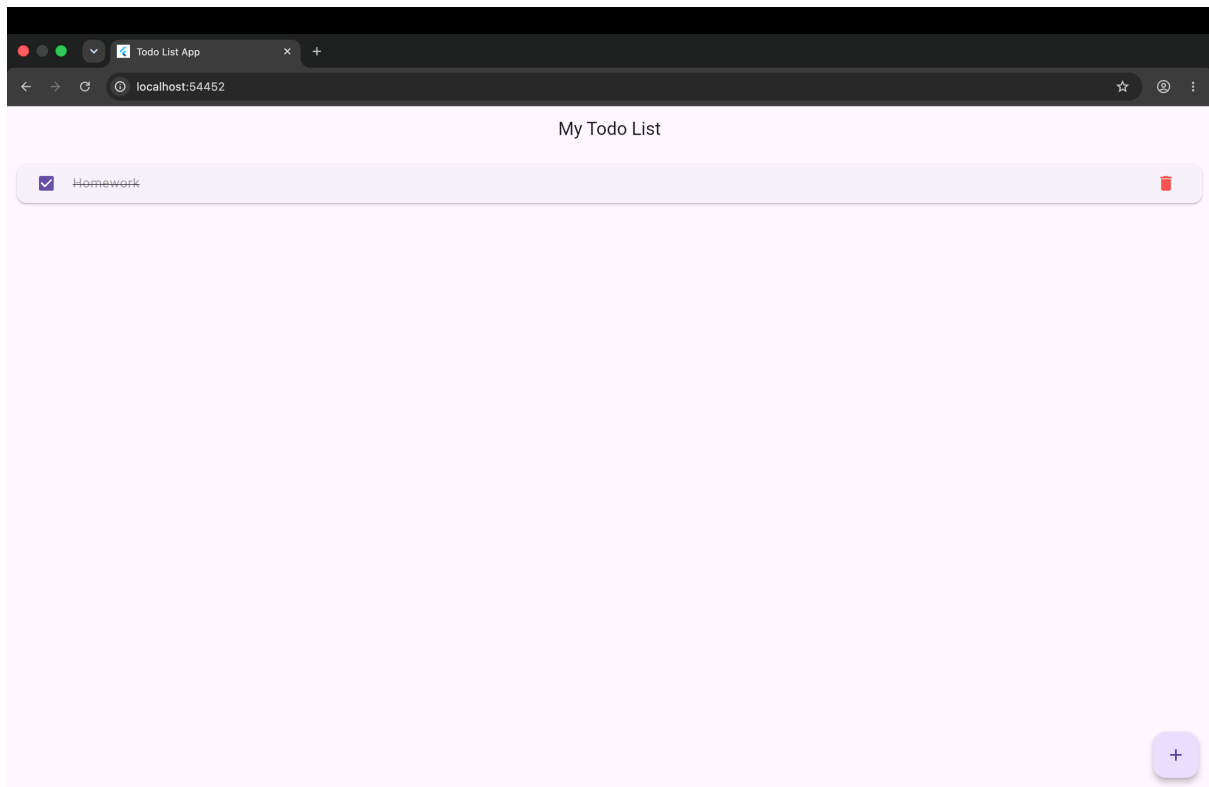
UI Output Verification

When launched via flutter run:

- An AppBar displays centered text titled **My Todo List**.
- The main screen renders a fallback text label (**No tasks added yet!**) when the `_todos` list is empty.
- Tapping the floating **+** button opens a modal bottom sheet containing a focused task input field and submission button.
- Adding items populates elevated Card widgets showing actionable checkboxes, titles, and red delete icons.
- Toggling a checkbox updates text styling to strike-through (`lineThrough`) and muted grey text, while pressing the trash icon instantly removes the target item.







Problems Encountered & Solutions Applied

Problem / Error	Cause	Resolution
Bottom sheet covered by software keyboard	Keyboard overlay obscuring TextField input area in showModalBottomSheet.	Wrapped content padding with <code>EdgeInsets.only(bottom: MediaQuery.of(ctx).viewInsets.bottom + 20)</code> and enabled <code>isScrollControlled: true</code> .
Memory leak from Controller	<code>TextEditingController</code> persisting in memory after widget removal.	Implemented explicit <code>_textController.dispose()</code> invocation inside state <code>dispose()</code> lifecycle method.
List rendering overflow	Dynamic task expansion causing bounded vertical spatial overflow.	Replaced standard layout components with dynamically virtualized <code>ListView.builder</code> .

Key Learning Outcomes

- **Reactive State Mechanics:** Understanding how `setState()` notifies the Flutter framework to re-execute `build()` routines for stateful UI reconciliation.
- **Modal Overlay Lifecycle:** Implementing interactive modal bottom sheets alongside `TextEditingController` management to capture user inputs cleanly.
- **Dynamic Collections Handling:** Mastering item creation, array modification, and targeted element deletion within reactive Flutter state trees.